

COMP 3311

DATABASE MANAGEMENT SYSTEMS

LECTURE 6

RELATIONAL DATABASE DESIGN: NORMALIZATION

NORMALIZATION

- Normalization uses functional dependencies to **decompose** (i.e., **break up**) an **unsatisfactory relation schema** into **fragments** (two or more relation schemas) that possess more desirable properties.

✎ **Eliminates data redundancy and update anomalies, preserves dependencies and is lossless.**

- Normalization provides a **series of tests** for relation schemas.

✎ **If a relation schema fails the test, then it needs to be decomposed.**

- Normalization is expressed in terms of normal forms.

✎ **We will normalize relations using only FDs.**

✎ **Normal forms do not guarantee a good design!**

NORMALIZATION: GOALS

Design Guideline 1: Clear Semantics for Attributes

Design a relation schema so that it is easy to explain its meaning. Typically, this means that we should not combine attributes from multiple real-world entities in a single relation schema.

✎ Each relation schema should have a well-defined, unambiguous meaning.

Design Guideline 2: Minimize Use of Null Values

As far as possible, avoid placing attributes whose values may be null in relation schemas. If nulls are unavoidable, make sure that they apply in exceptional cases only.

✎ Null values lead to problems of understanding the meaning of attributes and executing certain operations (e.g., sum, average, etc.).

NORMALIZATION: GOALS (CONTD)

Design Guideline 3: Minimize Redundancy

Design relation schemas so that **no operation** (insertion, deletion or update) **anomalies occur** in the relation instances. If any operation anomalies are present, note them clearly so that programs that change the database will operate correctly.

✎ **A relation schema has redundancy if there is an FD where the LHS is not a primary key.**

✎ **Redundant data in relations can cause operation anomalies.**

Operation Anomalies

- **insertion** (e.g., insert license fee for engine size 5)
✎ Recall entity integrity constraint
- **deletion** (e.g., delete instance “BMW, 7.35i, ...”)
- **update** (e.g., update license fee for engine size 1)

Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	fee	origin	tax
Nissan	Sunny	1	4,000	Japan	90
Fiat	Mirafiori	1	4,000	Italy	85
Honda	Accord	1	4,000	Japan	90
Toyota	Camry	4	7,000	Canada	50
Ford	Mustang	4	7,000	Canada	50
Ford	Mustang	2	5,000	U.S.A.	75
BMW	7.35i	3	6,000	Germany	95
Toyota	Camry	1	4,000	Japan	90

NORMALIZATION: GOALS (CONTD)

Design Guideline 4: Lossless Decomposition

The normalized relation schemas **should contain the same information** as the original schema; otherwise, decomposition results in information loss.

- A decomposition is **lossless** (aka **lossless join**) if the **original relation instance can be recovered** from the decomposed relations (the fragments).
 - Joining all the fragments (i.e., putting them back together into a single relation instance) results in **exactly** the original relation instance.
- In general, a decomposition of R into R_1 and R_2 is **lossless if and only if** at least one of the following FDs is in F^+ :

$$R_1 \cap R_2 \rightarrow R_1$$

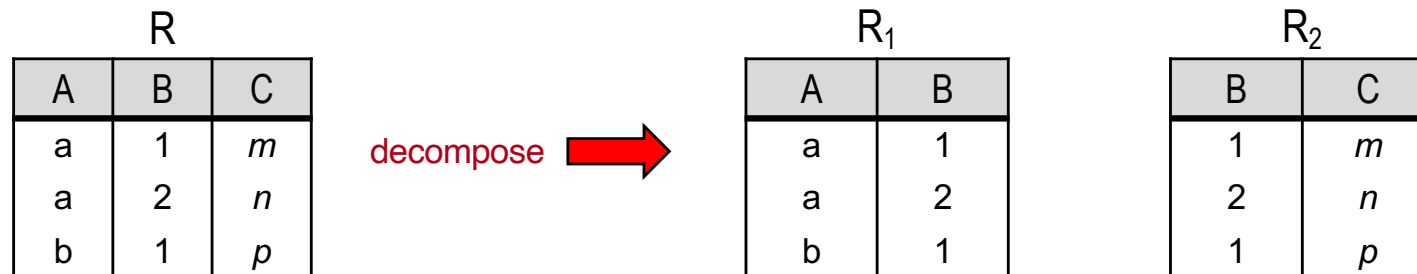
$$R_1 \cap R_2 \rightarrow R_2$$

👉 **The common attributes of R_1 and R_2 must be a superkey for R_1 or R_2 .**

NORMALIZATION: GOALS (CONTD)

Example: Lossy decomposition

Decompose $R(A, B, C)$ into $R_1(A, B)$ and $R_2(B, C)$.



R₁ JOIN R₂ on B

A	B	C
a	1	m
a	1	p
a	2	n
b	1	m
b	1	p

The *decomposition is lossy* since the join produces **two extra tuples**. Thus, the decomposition “loses” some information!
Note that the common attribute B of R₁ and R₂ is not a superkey of either R₁ or R₂.

NORMALIZATION: GOALS (CONTD)

Design Guideline 5: Preserve Functional Dependencies

As far as possible, functional dependencies should be preserved within each relation schema; otherwise, checking updates for violation of functional dependencies may require computing joins, which is expensive.

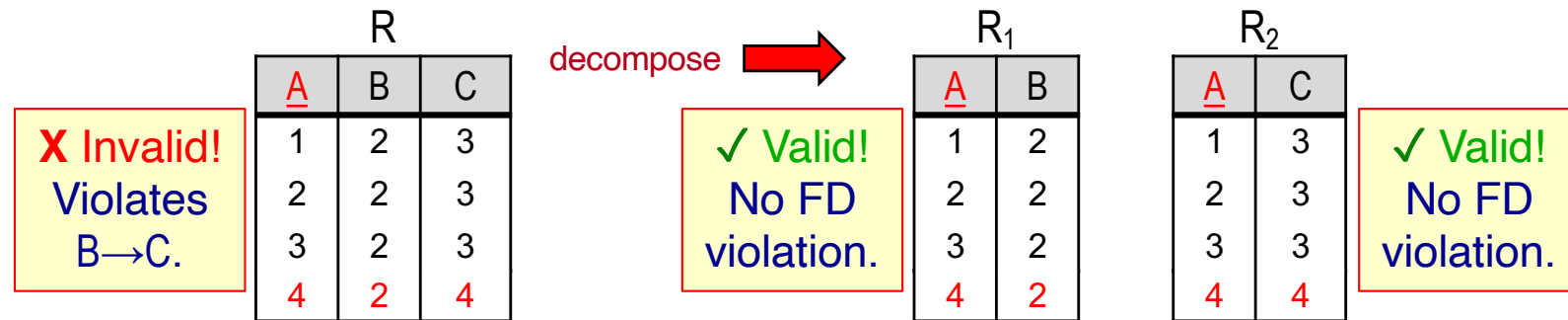
- Functional dependencies **represent real-world constraints**.
- If a functional dependency does not appear in any relation schema (i.e., it is “**lost**”), the **constraint may be much more difficult to enforce**.
- The decomposition of a relation schema R with FDs F is a set of schema fragments R_i with FDs F_i .
 - F_i is the subset of dependencies in F^+ (the closure of F) that involves only attributes in R_i .
- The decomposition is dependency preserving **if and only if** $(\cup F_i)^+ = F^+$.
 - Every FD in F^+ is present in some fragment R_i .

DECOMPOSITION EXAMPLE: NON-DEPENDENCY PRESERVING

$R(A, B, C)$ **Key:** A $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ $F^+ = F$

For FD $B \rightarrow C$, LHS is not a key $\Rightarrow$ R can have **considerable redundancy**.

Solution: Decompose R into relations $R_1(A, B)$ and $R_2(A, C)$.



The decomposition is **lossless** since the common attribute A is a key for R_1 (and R_2).

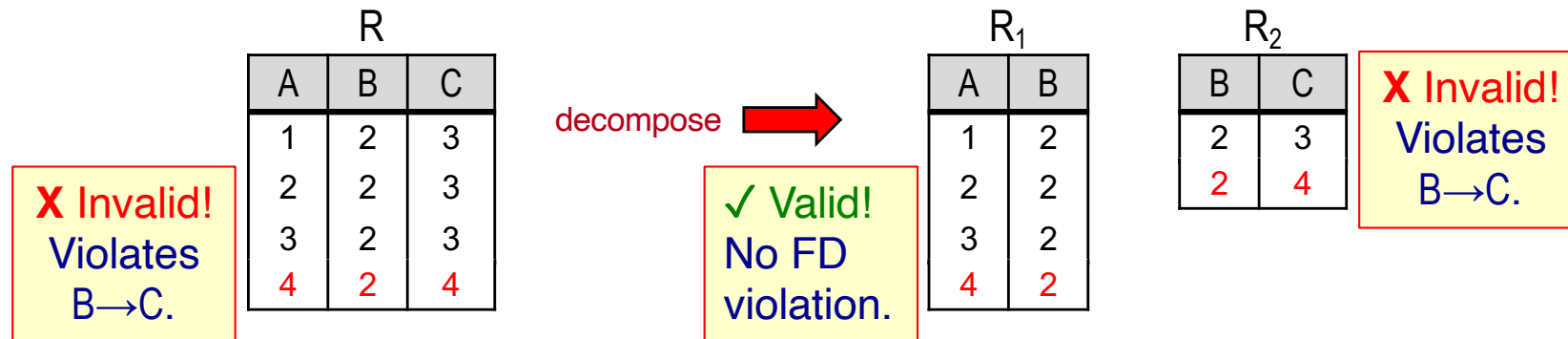
The decomposition is **not dependency preserving** because $F_1 = \{A \rightarrow B\}$, $F_2 = \{A \rightarrow C\}$ and $(F_1 \cup F_2)^+ \neq F^+$. **The FD $B \rightarrow C$ is lost.**

In practice, each “lost” FD is implemented as an **assertion**, which is checked when there are updates. Thus, to find violations on $B \rightarrow C$, R_1 and R_2 must be joined, which can be very expensive.

DECOMPOSITION EXAMPLE: DEPENDENCY PRESERVING

$R(A, B, C)$ **Key: A** $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ $F^+ = F$

Decompose R into relations $R_1(A, B)$ and $R_2(B, C)$.



The decomposition is **lossless** since the common attribute B is a key for R_2 .

The decomposition is **dependency preserving** because $F_1 = \{A \rightarrow B\}$, $F_2 = \{B \rightarrow C\}$ and $(F_1 \cup F_2)^+ = F^+$ since $A \rightarrow B$ and $B \rightarrow C \models A \rightarrow C$.

Violations of the FDs can be found by inspecting the individual relations, without performing a join.

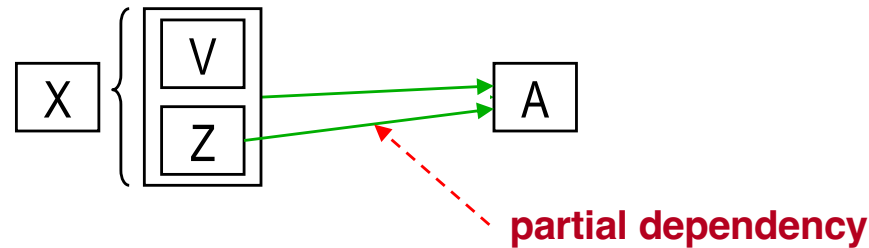
 **How a relation is decomposed, may determine whether functional dependencies are preserved.**

NORMAL FORMS:

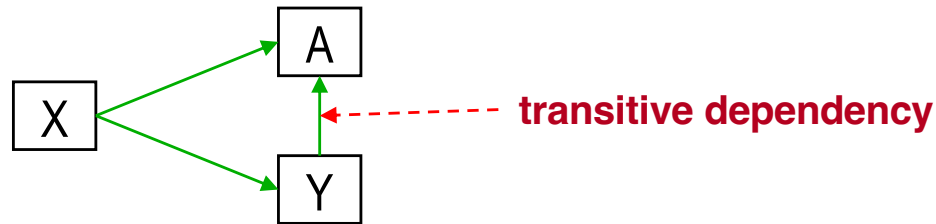
Prime and non-prime attributes

An attribute is a **prime attribute** if it is **part of any candidate key**. Otherwise, it is a **non-prime attribute**.

Partial dependency visualization



Transitive dependency visualization



NORMAL FORMS:

EXAMPLE RELATION SCHEMA & DATABASE

Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	fee	origin	tax
Nissan	Sunny	1	4,000	Japan	90
Fiat	Mirafiori	1	4,000	Italy	85
Honda	Accord	1	4,000	Japan	90
Toyota	Camry	4	7,000	Canada	50
Ford	Mustang	4	7,000	Canada	50
Ford	Mustang	2	5,000	U.S.A.	75
BMW	7.35i	3	6,000	Germany	95
Toyota	Camry	1	4,000	Japan	90

Functional Dependencies

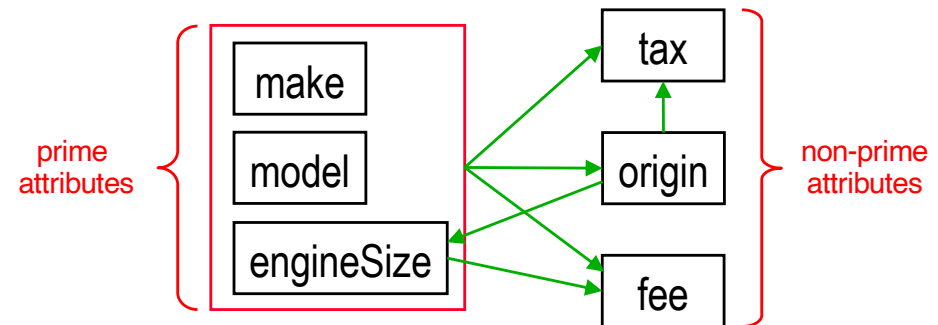
$\text{make, model, engineSize} \rightarrow \text{origin}$
 $\text{make, model, engineSize} \rightarrow \text{tax}$
 $\text{make, model, engineSize} \rightarrow \text{fee}$

} due to the primary key

$\text{origin} \rightarrow \text{tax}$
 $\text{engineSize} \rightarrow \text{fee}$
 $\text{origin} \rightarrow \text{engineSize}$

} from real-world knowledge

FD visualization



FIRST NORMAL FORM (1NF)

A relation schema is in *First Normal Form (1NF)* if all attributes are atomic (single-valued).

✎ There are **no multi-valued or composite attributes**.

- Relation schemas are always in 1NF according to the definition of the relational data model and according to our strategy for reducing an E-R schema to relation schemas.

SECOND NORMAL FORM (2NF)

A relation schema is in **Second Normal Form (2NF)** if **all non-prime attributes are fully functionally dependent on every candidate key.**

- **R** is a relation schema, with the set F of FDs.
- **R** is in 2NF *if and only if*

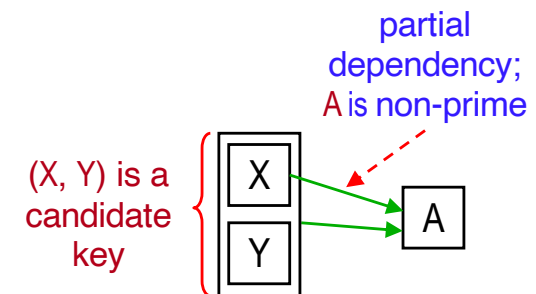
For each FD: $X \rightarrow A$ in F^+ :

$A \in X$ (the FD is trivial) **or**

(for LHS) X is **not a proper subset** of a candidate key for **R** **or**

(for RHS) A is a **prime attribute** for **R**.

✎ **A subset of a candidate key cannot determine a non-prime attribute.**



SECOND NORMAL FORM (2NF) EXAMPLE

- **make**, **model**, **engineSize** is a candidate key (it is not a proper subset).
- **engineSize** is a proper subset of a candidate key.
- **fee** is a non-prime attribute.
- Hence, the relation schema is **not in 2NF** due to the FD **engineSize**→**fee**.

Note redundancy.

Car

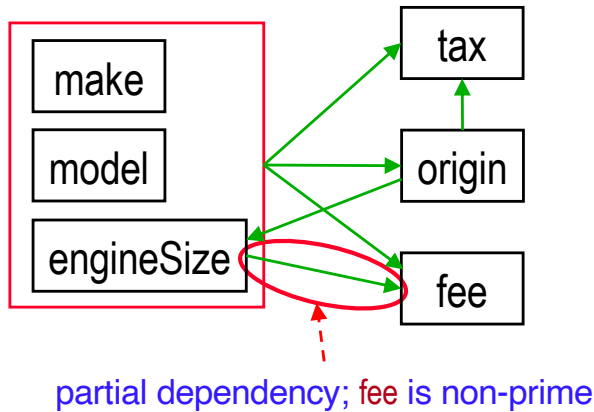
<u>make</u>	<u>model</u>	<u>engineSize</u>	fee	origin	tax
Nissan	Sunny	1	4,000	Japan	90
Fiat	Mirafiori	1	4,000	Italy	85
Honda	Accord	1	4,000	Japan	90
Toyota	Camry	4	7,000	Canada	50
Ford	Mustang	4	7,000	Canada	50
Ford	Mustang	2	5,000	U.S.A.	75
BMW	7.35i	3	6,000	Germany	95
Toyota	Camry	1	4,000	Japan	90

Recall Operation Anomalies

- **insertion** (e.g., insert license fee for engine size 5)
- **deletion** (e.g., delete instance “BMW, 7.35i, ...”)
- **update** (e.g., update license fee for engine size 1)

SECOND NORMAL FORM (2NF) EXAMPLE (CONT'D)

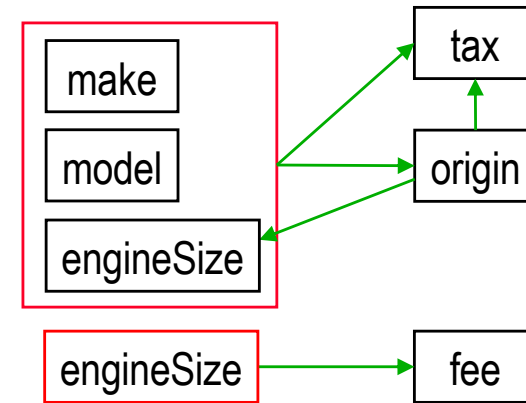
FDs in original schema



decompose
the schema



FDs in 2NF schemas



Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	origin	tax
Nissan	Sunny	1	Japan	90
Fiat	Mirafiori	1	Italy	85
Honda	Accord	1	Japan	90
Toyota	Camry	4	Canada	50
Ford	Mustang	4	Canada	50
Ford	Mustang	2	U.S.A.	75
BMW	7.35i	3	Germany	95
Toyota	Camry	1	Japan	90

Licensing

<u>engineSize</u>	fee
1	4000
2	5000
3	6000
4	7000

decompose
the relations



The decomposition
resolves the
previous operation
anomalies.



THIRD NORMAL FORM (3NF)

A relation schema is in *Third Normal Form (3NF)* if it is in 2NF and every non-prime attribute is non-transitively dependent on every candidate key.

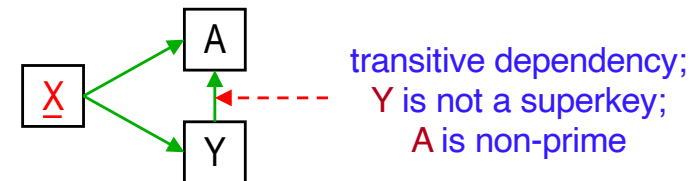
- **R** is a relation schema, with set *F* of FDs.
- **R** is in 3NF *if and only if*

For each FD: $X \rightarrow A$ in F^+ :

$A \in X$ (*trivial FD*) **or**

(for LHS) **X** is a **superkey** for **R** **or**

(for RHS) **A** is a **prime attribute** for **R**.



✎ **For every FD that does not contain extraneous attributes either:**

- the LHS is a candidate key, **or**
- the RHS is a prime attribute (i.e., it is part of some candidate key).

THIRD NORMAL FORM (3NF) EXAMPLE

- For the FD $\text{origin} \rightarrow \text{tax}$, origin is not a superkey.
- tax is not a prime attribute.
- Hence, the relation schema is **not in 3NF** due to the FD $\text{origin} \rightarrow \text{tax}$.

Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	origin	tax
Nissan	Sunny	1	Japan	90
Fiat	Mirafiori	1	Italy	85
Honda	Accord	1	Japan	90
Toyota	Camry	4	Canada	50
Ford	Mustang	4	Canada	50
Ford	Mustang	2	U.S.A.	75
BMW	7.35i	3	Germany	95
Toyota	Camry	1	Japan	90

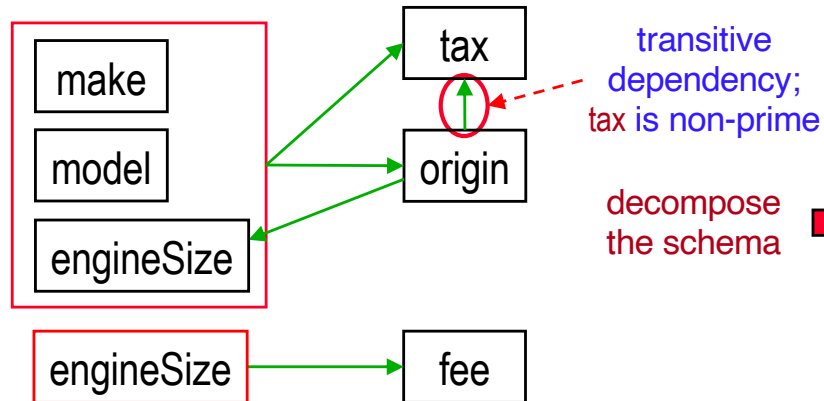
Note redundancy.

Operation Anomalies

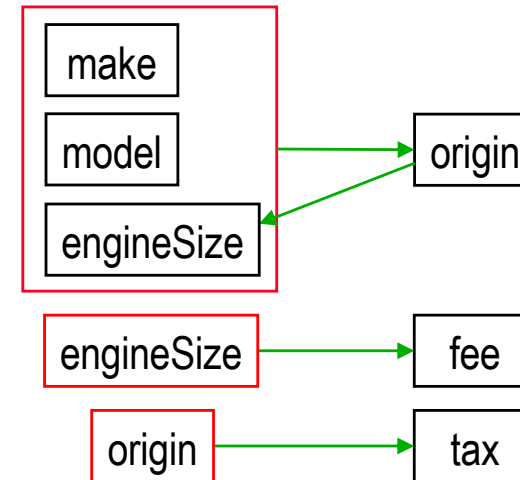
- **insertion** (e.g., insert tax rate 40 for Australia)
- **deletion** (e.g., delete instance “BMW, 7.35i, ...”)
- **update** (e.g., update tax rate for origin Japan)

THIRD NORMAL FORM (3NF) EXAMPLE (CONTD)

FDs in 2NF schemas



FDs in 3NF schemas



Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	origin
Nissan	Sunny	1	Japan
Fiat	Mirafiori	1	Italy
Honda	Accord	1	Japan
Toyota	Camry	4	Canada
Ford	Mustang	4	Canada
Ford	Mustang	2	U.S.A.
BMW	7.35i	3	Germany
Toyota	Camry	1	Japan

ImportTax

<u>origin</u>	tax
Canada	50
U.S.A.	75
Italy	85
Japan	90
Germany	95

Licensing

<u>engineSize</u>	fee
1	4000
2	5000
3	6000
4	7000

decompose the relations

The decomposition resolves the previous operation anomalies.

3NF Decomposition Rule

If none of the decomposed relation schemas contains a candidate key of the original relation schema, then add a relation schema containing one of the candidate keys.

3NF DECOMPOSITION ALGORITHM

Let **R** be the initial relation schema with FDs F .

Compute a **canonical cover** F_c of F .

$S = \emptyset$ (S is a set of relation schemas)

For each FD $X \rightarrow Y$ in the canonical cover F_c

$S = S \cup (X, Y)$ (for each FD create a relation schema; add it to S)

If no schema contains a candidate key for **R**

Choose any candidate key **K**

$S = S \cup K$ (add any candidate key as a relation schema)

✎ **The algorithm always creates a lossless-join, dependency preserving, 3NF decomposition.**

(Also called the 3NF Synthesis Algorithm.)

BOYCE-CODD NORMAL FORM (BCNF)

A relation schema is in *Boyce-Codd Normal Form (BCNF)* if every determinant (left hand side) of its FDs is a superkey.

- **R** is a relation schema, with the set **F** of FDs.
- **R** is in BCNF *if and only if*

For each FD: $X \rightarrow A$ in F^+ :

$A \in X$ (*trivial FD*) **or**

X is a **superkey** for **R**.

✎ **For every FD that does not contain extraneous attributes, the LHS is a candidate key.**

- BCNF relations have no redundancy (that can be removed by FDs).
- If a relation is in BCNF it is also in 3NF (and 2NF and 1NF).

BOYCE-CODD NORMAL FORM (BCNF): **EXAMPLE**

- For the FD $\text{origin} \rightarrow \text{engineSize}$, origin is not a superkey.
- Hence, this relation schema is **not in BCNF** due to the FD $\text{origin} \rightarrow \text{engineSize}$.

Note redundancy.

Car

<u>make</u>	<u>model</u>	<u>engineSize</u>	origin
Nissan	Sunny	1	Japan
Fiat	Mirafiori	1	Italy
Honda	Accord	1	Japan
Toyota	Camry	4	Canada
Ford	Mustang	4	Canada
Ford	Mustang	2	U.S.A.
BMW	7.35i	3	Germany
Toyota	Camry	1	Japan

Note: We need to use **null** values if we want to represent an engine size and origin, but do not know the make and model.

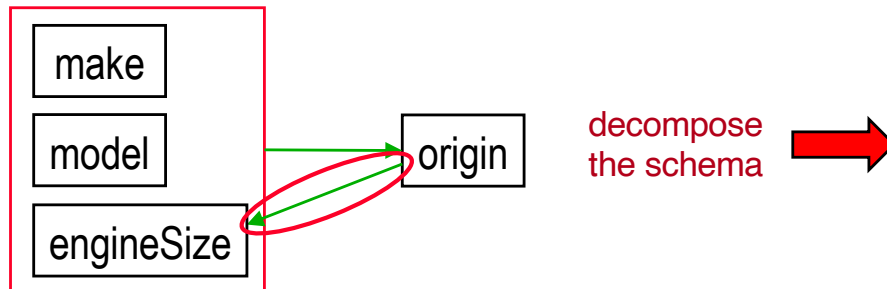
Operation Anomalies

- **insertion** (e.g., insert engine size 5 from Korea)
- **deletion** (e.g., delete all instances “Ford, Mustang, ...”)
- **update** (e.g., update engine size for Japan)

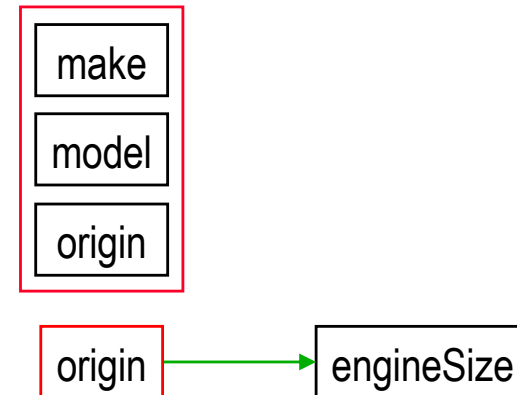
BOYCE-CODD NORMAL FORM (BCNF): **EXAMPLE**

(CONT'D)

FDs in 3NF schema



FDs in BCNF schemas



decompose the relations

The decomposition resolves the previous operation anomalies.

Car

<u>make</u>	<u>model</u>	<u>origin</u>
Nissan	Sunny	Japan
Fiat	Mirafiori	Italy
Honda	Accord	Japan
Toyota	Camry	Canada
Ford	Mustang	Canada
Ford	Mustang	U.S.A.
BMW	7.35i	Germany
Toyota	Camry	Japan

Country

<u>origin</u>	engineSize
Italy	1
Canada	4
U.S.A.	2
Germany	3
Japan	1

This decomposition avoids the 3NF problems of redundancy and null values.

BOYCE-CODD NORMAL FORM (BCNF): EXAMPLE (CONTD)

- We can generate the original relation instance by joining the two fragments using a **full outer join** (discussed in the next lecture).

Car			Country		Car			
<u>make</u>	<u>model</u>	<u>origin</u>			<u>make</u>	<u>model</u>	<u>engineSize</u>	origin
Nissan	Sunny	Japan		Italy	Nissan	Sunny	1	Japan
Fiat	Mirafiori	Italy		Canada	Fiat	Mirafiori	1	Italy
Honda	Accord	Japan		U.S.A.	Honda	Accord	1	Japan
Toyota	Camry	Canada		Germany	Toyota	Camry	4	Canada
Ford	Mustang	Canada		Japan	Ford	Mustang	4	Canada
Ford	Mustang	U.S.A.			Ford	Mustang	2	U.S.A.
BMW	7.35i	Germany			BMW	7.35i	3	Germany
Toyota	Camry	Japan			Toyota	Camry	1	Japan

Is the decomposition dependency preserving?

No We lose the FD $\text{make, model, engineSize} \rightarrow \text{origin}$.

A relation may not have a dependency preserving BCNF decomposition!

Can we have a dependency preserving decomposition?

No However, the relation schema is decomposed, $\text{make, model, engineSize} \rightarrow \text{origin}$ is lost since it involves all the attributes of the original 3NF Car relation.

BCNF DECOMPOSITION ALGORITHM

Let **R** be the initial relation schema with set of FDs F .

Compute F^+

$S = R$

Until all relation schemas in S are in BCNF

For each **R** in S

For each FD $X \rightarrow Y$ that violates BCNF for **R**

$S = (S - R) \cup (R - Y) \cup (X, Y)$

End until

- When a relation schema **R** with BCNF violation $X \rightarrow Y$ is found:
 1. Remove **R** from S , the set of relation schemas.
 2. Add a schema that has the same attributes as **R** except for Y (i.e., remove the RHS of the FD from **R**).
 3. Add a second schema that contains the attributes in X and Y (i.e., create a relation schema for the FD $X \rightarrow Y$).

BCNF DECOMPOSITION EXAMPLE

Given: $R(A, B, C, D, E)$ $F = \{A \rightarrow BE, C \rightarrow D\}$ (Note: $F = F^+$)

Candidate key: AC

- Both functional dependencies violate BCNF because the LHS is not a candidate key.
- Pick $A \rightarrow BE$
 - We can also choose $C \rightarrow D \Rightarrow$ different choices may lead to different decompositions.
- $R(A, B, C, D, E)$ generates:
 - $R_1(\underline{A}, C, D)$ (remove the RHS of $A \rightarrow BE$ from R)
 - $R_2(\underline{A}, B, E)$ (based on the FD $A \rightarrow BE$)

Do we need to decompose further? Yes Why?

BCNF DECOMPOSITION EXAMPLE (CONTD)

We have: $R_1(\underline{A}, \underline{C}, D)$ and $R_2(\underline{A}, B, E)$

$F = \{A \rightarrow BE, C \rightarrow D\}$ (Note: $F = F^+$)

Candidate key: AC

- We need to decompose $R_1(\underline{A}, \underline{C}, D)$ because of the FD $C \rightarrow D$.
- Thus $R_1(A, C, D)$ is replaced with $R_3(A, C)$ and $R_4(C, D)$.
- Final decomposition: $R_2(A, B, E)$, $R_3(A, C)$, $R_4(C, D)$.

BCNF DECOMPOSITION EXAMPLE (CONTD)

Final decomposition: $R_2(A, B, E)$ $R_3(A, C)$ $R_4(C, D)$
FDs: $F_2 = \{A \rightarrow BE\}$ $F_3 = \emptyset$ $F_4 = \{C \rightarrow D\}$

Is the decomposition lossless?

Yes the algorithm **always** creates lossless decompositions.

In step $S = (S - R) \cup (R - Y) \cup (X, Y)$ we replace **R** with relations $(R - Y)$ and (X, Y) that have **X** as the common attribute and $X \rightarrow Y$ (i.e., **X is the key of (X, Y)**).

Is the decomposition dependency preserving?

Yes because $(F_2 \cup F_3 \cup F_4)^+ = F^+$

 **But remember**, sometimes FDs cannot be preserved in a BCNF decomposition.

RELATIONAL DATABASE DESIGN: SUMMARY

- **Schema reduction** takes the data requirements represented using the E-R data model and reduces them to relation schemas that can be implemented by a commercial relational DBMS.
- **Functional dependencies** are **constraints** derived from the application domain that can be used to further **refine a relation schema** reduced from an E-R schema.
- When an E-R schema is **not well designed**, the relation schemas generated from it **may have undesirable properties** (**operation anomalies**). **Normalization** uses functional dependencies to remove these operation anomalies by **decomposing a relation schema** into normal forms.
 - While **BCNF is the "best"** normal form, it **may not be dependency preserving**.
 - There is always a dependency preserving 3NF decomposition and, in practice, **3NF is often "good enough"** for most applications.